Machine learning

Stepas Toliautas
10/15/26

Agents and information

- How to provide an agent with the information about its environment?
  - State space (state and transition data)
  - Parameters of Bayes network (probabilities)
- a) Input during the construction of an agent
  - Guaranteed to work, but not flexible
- b) Let the agent to shape sensor data for itself

- ***Machine learning*** concerns the estimation of model parameters or structured knowledge from provided data

Machine learning: vocabulary

- Learning types:
  - Supervised – unsupervised – reinforcement
- Teaching material:
  - Prepared data – raw data – sensor (environment reaction) data
- Learning outcomes:
  - Model parameters – data structure – hidden relations
- Learning goals:
  - Diagnosis – prognosis – generalization – autonomy
- Learning conditions:
  - Passive – active – real-time

Supervised learning

- x_m, y_m – *training* data →
- x_t, y_t – *control* data
  - Testing and/ or validation

$$\begin{bmatrix} x_{11} & x_{12} & \dots & x_{1N} & \rightarrow & y_1 \\ x_{21} & x_{22} & \dots & x_{2N} & \rightarrow & y_2 \\ & & \vdots & & & \\ x_{M1} & x_{M2} & \dots & x_{MN} & \rightarrow & y_M \end{bmatrix}$$

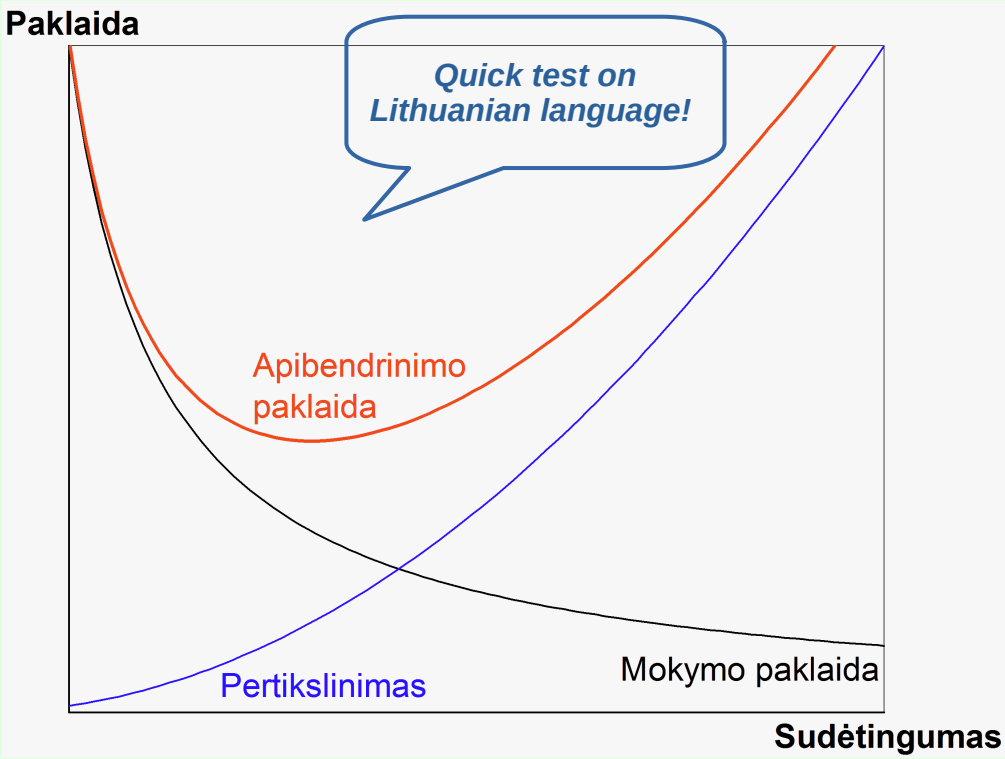
What we get: $f(\vec{x}_m) \approx y_m$

What we check: $f(\vec{x}_t) \rightarrow y_t$

- **The goal** – to find function $y = f(x)$
  - Expecting: f – actual relations between x and y
  - Checking: f that most ~~accurately~~ sensibly models x_t, y_t
  - f is generally ***not supposed*** to be perfect fit for x_m, y_m!

Errors in learning process

- More complex models:
  - Better fit x_m, y_m
  - Are slower to compute
  - Can *overfit* data
- Balance is the key
  - Accuracy ↔ complexity
  - *Learning* error is calculated directly from data
  - *Generalization* error is harder to estimate



Classification: spam e-mail

100% customer satisfaction is what everyone at our online pharmacy is dreaming about!

- Not all e-mails are worth reading...
- Supervised learning – great technique for spam filters
  - Plentiful training data (just create an e-mail inbox)
  - Well-separated classes (specific, limited vocabulary)
    - And when spammers get smarter, so do spam filters

Classification model: bag-of-words

- Data is provided as *texts*
  - Entire e-mails or single sentences
- Each text can be u*z*eful (*Z*) or *s*pam (*S*)
- Text elements – *words*
  - Position of words in text does not matter in this case
  - *Vocabulary size* – count of unique words in all texts (*corpus*)
- *Prior* probabilities:
  - Proportion of spam texts, $P(S)$
  - Word frequencies in each text class, $P(W_i \mid Z \equiv \neg S)$, $P(W_i \mid S)$

Spam e-mail: data

- Real:
  - PLAY SPORTS TODAY
  - WENT PLAY SPORTS
  - SECRET SPORTS EVENT
  - SPORTS IS TODAY
  - SPORTS COSTS MONEY
- Spam:
  - OFFER IS SECRET
  - CLICK SECRET LINK
  - SECRET SPORTS LINK

- What is the size of the vocabulary?
- What is the probability for random text to be spam?

Maximum likelihood „method“

What is the probability for randomly chosen text to be spam?
Let separate texts be *independent* from the *same distribution*.
We look for probability $P(S) = \pi$,
so that joint probability $P_{\text{data}}(SSSZZZZZ)$ is largest.

$$P(t_i) = \begin{cases} \pi, \text{ if } t_i = S, \\ 1-\pi, \text{ if } t_i = Z. \end{cases}$$

$$P_{\text{data}} = \prod_i P(t_i) = \pi^{N(t=S)} \cdot (1-\pi)^{N(t=Z)} = \pi^3 \cdot (1-\pi)^5;$$

$$\ln P_{\text{data}} = 3\ln\pi + 5\ln(1-\pi); \quad \frac{\mathrm{d}\ln P_{\text{data}}}{\mathrm{d}\pi} = \frac{3}{\pi} - \frac{5}{(1-\pi)} = 0;$$

$$5\pi = 3(1-\pi),\ 8\pi = 3,\ \pi = 3/8.$$

Learning and using model parameters

- Calculate most likely (ML) values for probabilities:
  - $P(\text{SECRET} \mid S)$
    - 3/9 = 1/3
  - $P(\text{SECRET} \mid Z)$
    - 1/15
- Using obtained model, estimate the following:
  - $P(S \mid \text{SPORTS})$
    - [*Bayes theorem*] = 3/3 / (3/3 + 5/1) = 1/6
  - $P(S \mid \text{SECRET IS SECRET})$
    - $3/(9 \cdot 9) / [3/(9 \cdot 9) + 5/15^3] = 25/26$
  - $P(S \mid \text{TODAY IS SECRET})$
    - $0 \cdot \ldots / (0 \cdot \ldots + \ldots) = 0$!
- …so how the words not in data *should* be treated?

Smoothing

- Training data is not the absolute truth
  - *Smoothing* of the distribution somewhat protects it from noise
- *Laplace* smoothing
  - C – count of different values a variable can take
    - In our example, for texts $C_{\text{text}} = 2$ (*S* or *Z*),
      for words $C_{\text{word}} = 12$ (vocabulary size)
  - k – smoothing parameter (a case of *hyperparameter*)
    - Best k value is found by optimizing learning process itself

$$\text{ML:} \quad P(x) = \frac{N(x)}{N_{\text{tot}}}$$

$$\text{LS:} \quad P(x) = \frac{N(x) + k}{N_{\text{tot}} + k\,C_x}$$

Laplace smoothing exercise

- Let $k = 1$
- Calculate the following probabilities:
  - $P(S)$, $P(Z)$
  - $P(\text{TODAY} \mid S)$, $P(\text{TODAY} \mid Z)$
  - $P(S \mid \text{TODAY IS SECRET})$

Classification in continuous space

- *Separator* functions are used
  - Because they divide parameter space (separate classes)
- Simplest linear separator – *perceptron*:
  - $P(x) = \begin{cases} 1, \text{ if } w_i x + w_0 \geqslant 0 \\ 0, \text{ if } w_i x + w_0 < 0 \end{cases}$

Training algorithm:

$w_i^0,\ w_0^0$ – small random values or zeros

$w_i^m = w_i^{m-1} + \alpha\left(y_j - f\left(x_j\right)\right) \cdot x_{ji}$

α – learning rate

Logistic „regression“

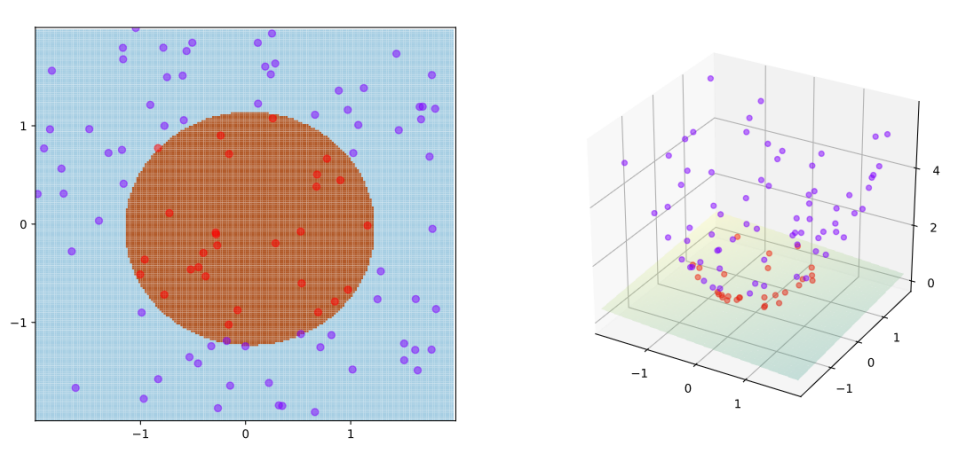
- Linear classification has a disadvantage
  - Hard category (class) assignment, expressed as *step function*
  - Awkward to use for symbolic analysis (e. g. analytic derivatives)

- It is possible to use *continuous* threshold function
  - Class probability – a number between 0 and 1
  - Boundary-adjacent data retain their „fuzzy“ assignment during learning
  - More stable in cases of noise and realistic data
  - Using a threshold function below is called *logistic regression*:
  - $$Z = \frac{1}{1 + e^{-f(x)}}$$

Advanced classification

- Data can be perfectly divided by multiple functions
  - ***Best*** separator = largest parameter gap between categories (classes)
  - Most resistance to future noise is ensured

- Non-linear classification is much harder
  - ...unless you can cleverly re-map the input data (*kernel trick*)
  - Optimal separator + kernel trick → SVM (*support vector machines*)



Non-parametric methods

- Separator is defined by a fixed number of parameters
  - Best if form of separation boundary is approximately known
  - Misses the opportunity to finely shape boundary by incoming data

- *k-neighbors*, a. k. a. *nearest-neighbors*
  - Data point is classified to the majority of nearest neighbors
  - Best correspondence is ensured by optimizing for different *k* values
  - *k* also works as smoothing parameter
    - Few neighbors – jagged separation, individual points seem „important“
    - Many neighbors – smoothed boundaries, less weight for individual points

Cross-validation

- How to optimize smoothing parameters?

- Divide data into three parts
  - f. e., 80 % training, 10 % validation, 10 % testing
- *Train* using different *k*, use *validation* data to select the best
  - Maybe even extrapolate local minimum
- Re-train with the best *k* and *test* using the last 10 %
  - There are ways to mix and reuse training/ validation data
  - However, final test must use not-seen-before data set

Regression: learning continuous functions

- $y = f(x)$ – continuous relation
  - Linear regression: $f(x) = w_1 x + w_0$

- How to define „the most suitable“ f ?
- By the minimization of *loss function*
  - Example – least-squares:

  - $L_{\mathrm{LS}} = \sum_i (y_i - w_1 x_i - w_0)^2$

Linear least squares

a) $$\frac{\partial L}{\partial w_0} = -2\sum_{i=1}^{M}\left(y_i - w_1 x_i - w_0\right) = 0;$$

$$\sum_i y_i - w_1 \sum_i x_i = M w_0,$$

$$\boldsymbol{w_0} = \frac{1}{M}\sum_i y_i - \frac{w_1}{M}\sum_i x_i$$

b) $$\frac{\partial L}{\partial w_1} = -2\sum_{i=1}^{M}\left(y_i - w_1 x_i - w_0\right)\cdot x_i = 0;$$

$$\boldsymbol{w_1} = \frac{M\sum_i x_i y_i - \sum_i x_i \sum_i y_i}{M\sum_i x_i^2 - \left(\sum_i x_i\right)^2}$$

(*scribble* *scribble*)

Complexity and regularization

- Reminder: models are subject to contradictory demands
  - *Simplicity* (least number of parameters, class labels etc.)
  - *Accuracy* and *flexibility* in describing available data
- *Regularization* is a way to reconcile the two
  - Loss function is supplemented with complexity term:
    $L = L_{\text{data}} + L_{\text{param}}$
  - L_{data} can be least-squares error
  - L_{param} can be a weighted sum of parameters: $\Sigma_i |w_i|^p$, here $p = 1, 2, \ldots$
  - $p = 1$ tends to zero-out redundant parameters
  - $p = 2$ avoids anisotropy (favored direction) in parameter space
- Now analytic solutions are usually impossible to find
  - Minimization of total loss function is performed numerically

Unsupervised learning

- Uses real-world, raw data
- „Correct“ answers are not known in advance
- **The goal** – to find useful relations, trends, insights

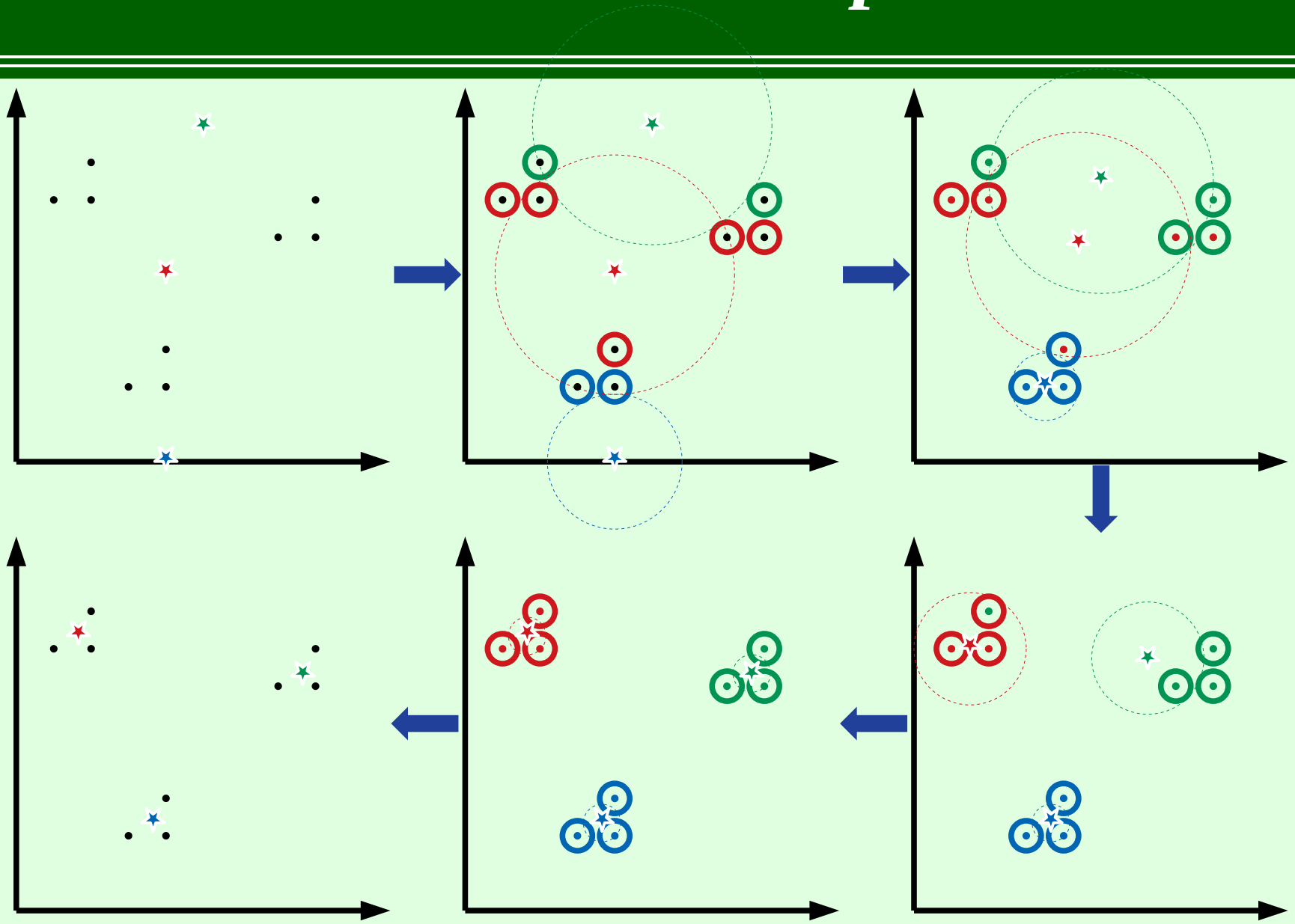
- Applications:
- Probability analysis
  - Pattern search
  - Parameter space reduction
- Factorization
  - „Blind“ signal separation

Clustering: k-means

- Search for most probable data grouping
  - Initial coordinates of cluster centers are chosen
  - Data points are assigned to clusters based on ***distance***
  - Cluster centers are moved to the geometric or mass center of groups
  - Data assignment and cluster relocation is repeated to convergence
  - Empty cluster centers are removed or randomly regenerated
- Result – local optimum of cluster centers

- Main drawbacks (all related to distance criterion):
  - Clusters of similar size
  - Clusters of spherical shape
  - Boundaries tend to be equidistant from the cluster centers

k-means: example



Reminder: Gaussian distribution

a) **1D** Gaussian: $f\left(x|\mu,\sigma^2\right)=\frac{1}{\sqrt{2\pi\sigma^2}}\mathrm{e}^{\frac{\left(x-\mu\right)^2}{2\sigma^2}}$

Parameters μ, σ^2: $\mu_{\text{data}}=\frac{1}{M}\sum_{i=1}^{M}x_i,\quad \sigma^2_{\text{data}}=\frac{1}{M}\sum_{i=1}^{M}\left(x_i-\mu_{\text{data}}\right)^2$

(Can be derived using maximum likelihood rules.)

b) **Multivariate** Gaussian: $f_N\left(\vec{x}|\vec{\mu},\boldsymbol{S}\right)=\frac{1}{\sqrt{\left(2\pi\right)^N}}|\boldsymbol{S}|^{-\frac{1}{2}}\mathrm{e}^{-\frac{1}{2}\cdot\left(\vec{x}-\vec{\mu}\right)^{\mathrm{T}}\boldsymbol{S}^{-1}\left(\vec{x}-\vec{\mu}\right)}$

Parameters $\vec{\mu}$, $\boldsymbol{S}$: $\vec{\mu}_{\text{data}}=\frac{1}{M}\sum_{i=1}^{M}\vec{x}_i,\quad \boldsymbol{S}_{\text{data}}=\frac{1}{M}\sum_{i=1}^{M}\left(\vec{x}_i-\vec{\mu}_{\text{data}}\right)^{\mathrm{T}}\left(\vec{x}_i-\vec{\mu}_{\text{data}}\right)$

Expectation maximization

- Generalization of *k*-means
  - Points are grouped by the *most probable* of cluster assignments
  - Clusters themselves are multivariate *Gaussian PDFs*:
    - Point probability $P(\vec{x}) = \sum_{i=1}^{k} p(C=i) \cdot P(\vec{x}|C=i)$, where

      $p(C=i) \equiv \pi_i$ – probability of assignment to the i th cluster,

      $P(\vec{x}|C=i)$ – i th cluster Gaussian value at $\vec{x}$ using $\vec{\mu}_i, \boldsymbol{S}_i$

      E-step: fixed π_i, $\vec{\mu}_i$, $\boldsymbol{S}_i$

      $$l_{ij} = \pi_i \cdot (2\pi)^{-N/2} |\boldsymbol{S}_i|^{-1/2} \mathrm{e}^{-1/2 \cdot (\vec{x}_j - \vec{\mu}_i)^{\mathrm{T}} \boldsymbol{S}_i^{-1} (\vec{x}_j - \vec{\mu}_i)}$$

      M-step: recalculation

      $$\pi_i = \frac{1}{M} \sum_j l_{ij}, \quad \vec{\mu}_i = \frac{\sum_j l_{ij} \vec{x}_j}{\sum_j l_{ij}}, \quad \boldsymbol{S}_i = \frac{\sum_j l_{ij} (\vec{x}_j - \vec{\mu}_i)^{\mathrm{T}} (\vec{x}_j - \vec{\mu}_i)}{\sum_j l_{ij}}$$

Number of clusters

- Empty clusters can be regenerated
- ...or their number k can be optimized
  - The following functional form is commonly used:
  - $-\sum_j \ln P\left(x_j|\mu,\sigma^2,k\right) + \lambda\cdot k$
  - Algorithm can be repeated from different initial positions
    - By moving all or the weakest cluster centers
  - This way, problem of local minima is mitigated

Dimension reduction

- Simpler model is created from the complex one
  - By ignoring least significant parameters
- Linear dimension reduction:
  - Data fitted as multivariate Gaussian
  - Eigenvalues and eigenvectors of covariant matrix **S** are computed
  - Largest eigenvalue are selected
  - Data is projected along the corresponding eigenvectors

Dimension reduction: example

Data:

$$\boldsymbol{x} = \begin{pmatrix} 0 & 1{,}9 \\ 1 & 3{,}1 \\ 2 & 4 \\ 3 & 5{,}1 \\ 4 & 5{,}9 \end{pmatrix}$$

Find eigenvalues/ eigenvectors of $\boldsymbol{S}$. What is the approximate dimension of data?

$\vec{\mu} = \left((0+1+2+3+4)/5,\ (1{,}9+3{,}1+4+5{,}1+5{,}9)/5\right) = (2, 4)$

$S_{11} = \left((-2)^2+(-1)^2+0+1^2+2^2\right)/5 = 2$

$S_{12} = \left(-2\cdot-2{,}1-1\cdot-0{,}9+0\cdot0+1\cdot1{,}1+2\cdot1{,}9\right)/5 = 2$

$S_{22} = \left((-2{,}1)^2+(-0{,}9)^2+0^2+1{,}1^2+1{,}9^2\right)/5 = 2{,}008$

Eigenvalues: $\boldsymbol{S} = \begin{pmatrix} 2-\lambda & 2 \\ 2 & 2{,}008-\lambda \end{pmatrix}$;

$\lambda^2 - 4{,}008\,\lambda + 0{,}016 = 0;\quad \lambda_{1,2} = 4{,}004,\ \mathbf{0{,}004}$ **!**

Eigenvectors: $\begin{vmatrix} 2 & 2 \\ 2 & 2{,}008 \end{vmatrix} \begin{vmatrix} x \\ y \end{vmatrix} = \lambda \begin{vmatrix} x \\ y \end{vmatrix}$;

4,004: $y = 1{,}002\,x,\ x^2 + y^2 = 1,\ x = \sqrt{1/2{,}004}$,

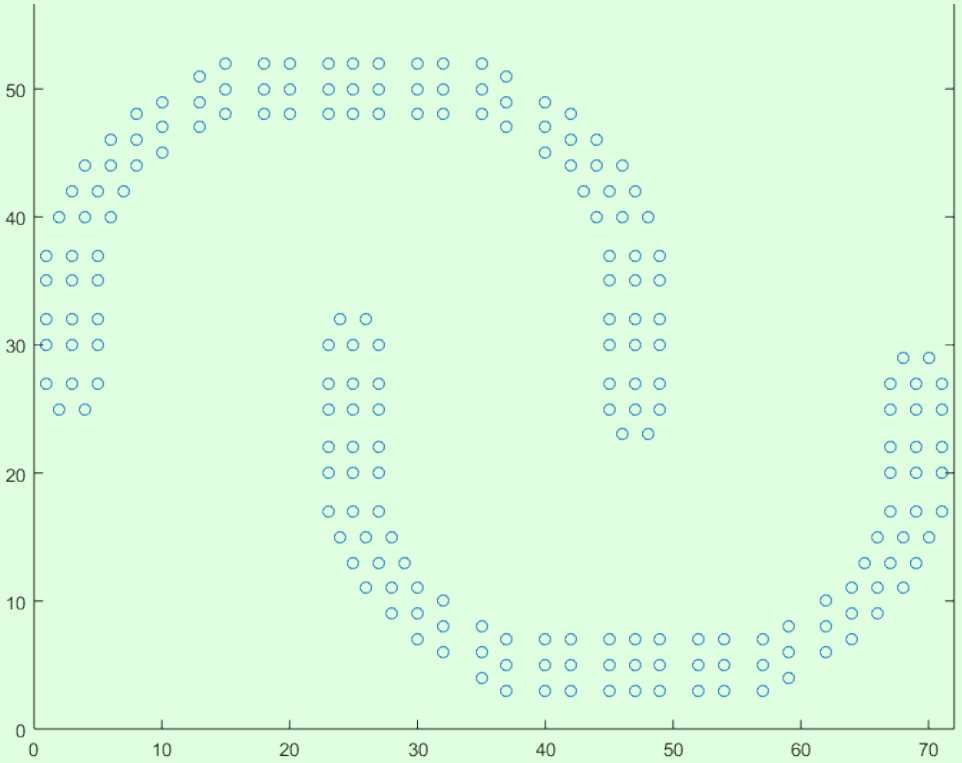
$(x, y)_{4{,}004} = (0{,}7064, 0{,}7078)$;

$(x, y)_{0{,}004} = (-0{,}7078, 0{,}7064)$.

Nonlocal structure search

- How many groups are here?
- Do you expect *k*-means/ EM to work in this case?
  - Why?

- How can data *affinity* be described in such cases?

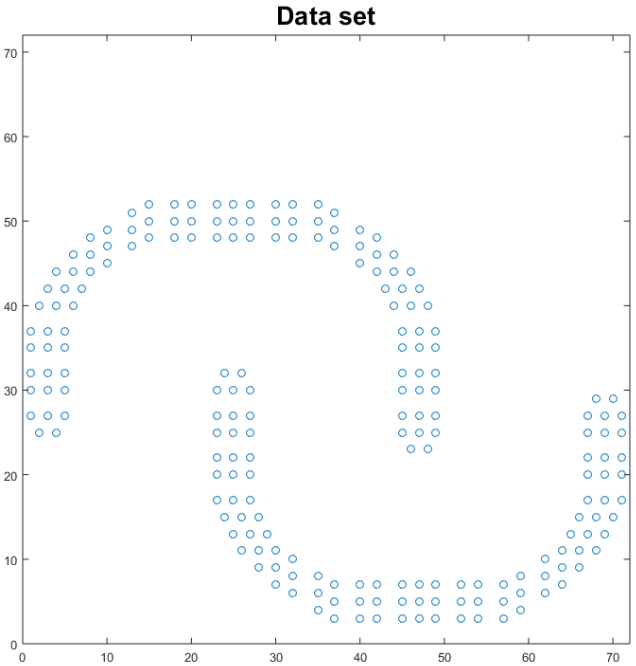


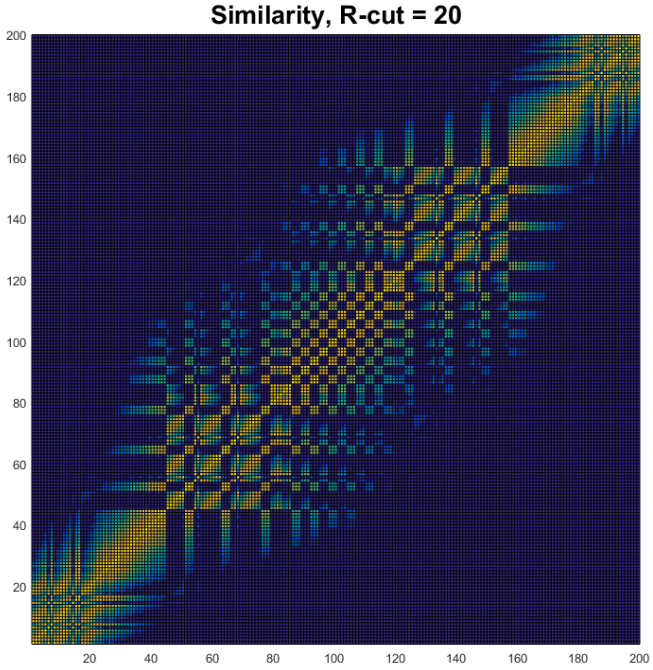
Spectral clustering

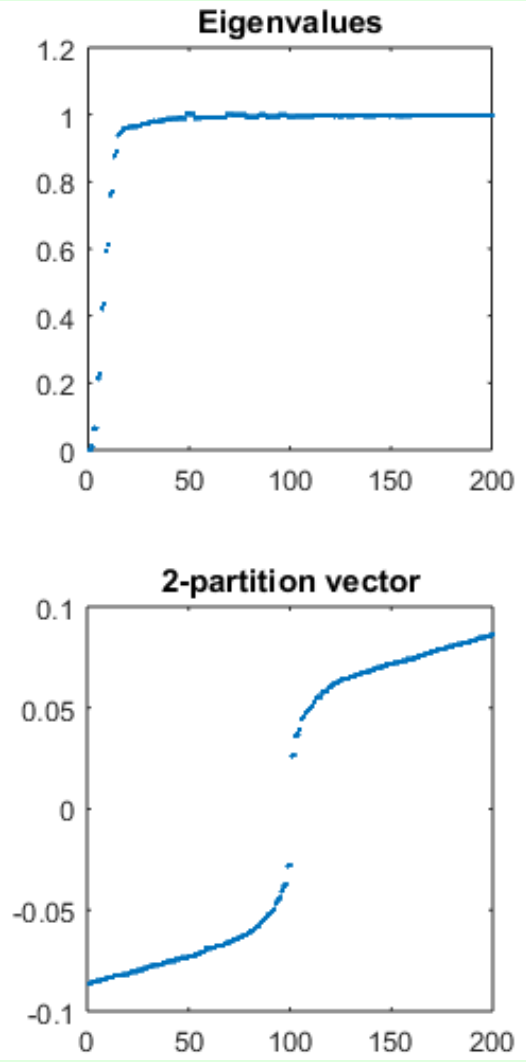
- Choice of *similarity matrix* is made
  - Distances, covariant matrix, pair correlation, ...
- Eigenvalue problem is solved
  - Or SVD, or spectral components, ...
- Data is projected to the most significant eigenvectors

- One example of solution procedure:
  - *https://calculatedcontent.com/2012/10/09/spectral-clustering/*

Spectral clustering: similarity







Spectral clustering: class labeling

